

第五题知识点整理

- 本小登无任何Python基础，本题绝大部分由 AI 辅助学习，另外还观看了部分题目中给出的B站教学视频合集、查阅了相关资料。
- 源代码、学习笔记所有内容均为古法手搓，不存在任何复制粘贴情况
- 与 GPT 的对话链接
 - <https://chatgpt.com/share/6ab100e2-746c-83ee-b4d9-305c1ba23db8>
 - <https://chatgpt.com/share/6ab2aa02-acec-83e8-bec8-97ce625b1b72>
- 本小登的在学习代码的过程中随笔写下这篇知识点笔记，完成代码后，借助 Chat GPT 梳理了知识的分类，然后对知识点进行分类和汇总。

1. 机器学习与数据基本概念

1.1 数据与字段

- **CSV**：一种保存表格数据的 **文本文件** 格式。
- **样本**：一条独立的数据记录。
- **特征**：用来描述样本、提供给模型、帮助模型预测的信息。
- **字段**：特征的类型。
- **标签**：数据中给出的结果/模型预测的结果。
- **缺失值**：某个样本的某个值本应该被观测或者记录，但没有被提供、或者无法被获得，这个值就是缺失值。
- **非缺失值**：这一数据真正存有数据，而不是 **NaN** 等缺失状态。
- **数值特征**：本身就是数值，且这些数值的大小、差值通常具有实际意义。
- **类别特征**：表示属于“哪一类”的数据。
- **类别编码**：把类别信息转成能够进行数学计算的数值形式。
- **有序类别特征**：不同的类别之间存在顺序关系。
- **唯一标识符号**：用来区分不同记录的编号。每个乘客有自己的编号。
- **基数**：一个类别字段一共有多少种不同取值。
 - 如果一个字段的不同类别非常多，就叫高基数类别特征。

1.2 训练集、测试集与预处理

- **训练集**：真正拿来让模型学习的数据集。
- **测试集**：在训练之后用于检查模型泛化能力的数据集。
- **预处理**：再把原始数据交给模型之前，对数据进行整理、转换、编码等，使其便于模型学习和训练。
- **数据泄露**：提前把验证集、训练集中的数据泄露给模型，或者提供给模型的特征体现了预测结果等
- **平均值填补缺失**：当测试集中的数据缺失且需要填补时，应使用训练集的数据填补，而不是测试集。
 - **原因**：当预测新乘客时，不会事先拥有一整批“未来数据”来计算平均值。
- **预处理参数**：预处理过程中，在训练数据中计算出来的，以后需要保存、复用的数值或者规则，例如某个特征的平均值，某个类别的编码规则。
- **随机划分**：先随即打乱样本，再划分数据集、验证集、测试集等。
- **特征工程**：根据原始字段构造出一个可能更适合模型使用的新特征。
- **特征选择**：决定原始数据中的哪些字段真正作为模型输入。
- **目标泄露**：本来只有预测结束之后才能知道的信息，提前出现在了模型输入中。
- **数据尺度**：一个特征通常处在多大的数值范围内。

- **特征缩放**：调整不同数值特征的尺度，使它们跟适合模型训练。
- **std**: standard deviation, 标准差, 用来描述一组数据通常离平均值有多远, 也就是数据分散的程度。

1.3 随机性与可复现性

- **伪随机数生成器**：根据一种确定的数学算法来生成“看起来随机”的数学序列的程序。
- **随机种子**：控制随机过程的起点, 使随即划分一致、参数初始化一致等。
 - 常把伪随机数生成器初始化到由随机种子确定的状态, 然后从这个状态开始产生随机结果。此时用 **随机种子** 表示 **随机程度**, 只是一个用来初始化伪随机数生成器的值。
- **核心关系**：相同数据 + 相同随机算法 + 相同随机种子 ——→ 通常得到相同的随机结果。
- **可复现性**：再次运行实验时, 尽量得到相同的结果的性质。

2. Python 基础

2.1 Python、包、库与导入

- **库**：含有已经写好的大批函数、功能。
- **Python 包**：写好的一组 Python 功能。
- **import**：把某个库引入当前的 Python 程序。
- **as**：取一个简短的别名。
- **命名空间**：一个包自己的“名字仓库”。

2.2 变量、赋值与基本数据

- **变量**：某个数据的名字, 如 data。
- **=**：赋值符号。
- **字符串**：Python 中表示文字的一种数据。
- **数据类型**：一份数据在 Python/Pandas 看来属于什么种类, 因此可以进行什么样的计算。
- **布尔值**：**True(1)**、**False(0)**, 即专门表示“真/假”的数据。
- **None** Python 常见的缺失值。

2.3 Python 内置函数与常用功能

- **len()**：来自 **lenth (长度)**, 作用是计算一个对象中有多少项。
- **int()**：来自 **inter (整数)**, 作用是直接去掉小数部分。
- **range(start, end)** 生成一系列连续整数, 常用来控制 **for** 循环重复多少次。
 - range(start, end) 包含 start, 不包含 end。
 - range(1, 6) 表示 1, 2, 3, 4, 5
- **list()**：把某些可以依次读取的对象转换成列表。
- **input()** 在终端显示一段提示文字, 然后等待用户输入内容。
- **printf ()**：Python 自带的一个函数, 作用是把结果显示到终端上。
- **print() 多个输出的换行**：最后加入 sep = “\n\n”。
- **print ()** 最后一个数据后有无 “,” 均可。
- **()**：交给函数的信息。
- **zip()** 将两个序列按位置一一对应。
 - **for feature, error_message in zip(features, error):**
 - features, error 是两个等长列表。

2.4 Python 数据结构

- **列表**：Python 中按照顺序保存多个数据的一种结构。
- **append()** 把一个新元素添加到列表最后。
- **元组**：按顺序装着多个数据的一种 Python 数据结构。
- **dict (字典)** 按“名字”保存数据的结构。
 - ``person = {"Age":20, "Sex":"male"}`
 - 左边叫做键（如"Age"，"Sex"），右边叫做值(如"male",20)
 - 字典["..."]，中括号内是一个键名，这个表达式可以=字典，这样就形成了一个嵌套字典。
 - `{}` 表示一个空字典。
 - **键 (key)** 用来找到某个数据的唯一名称。
 - 同一个字典中，键不能重复保存两个不同值。

2.5 索引、切片、循环与可迭代对象

- **切片**：从一串数据中取出一个连续范围。
 - `:` **712** 表示从最开始一直取到位置712之前。
 - **712** `:` 表示从位置712取到最后。
 - `:` 取开头，不取结尾，除非后面为空白。
- `[]`：Python 中常用于选择某个位置或某一部分数据。
- **循环**：让计算机把同一段操作重复执行多次。
- **in**：从这一组数据中依次取出元素。
- **缩进 ()**：表示哪些代码属于这个循环。
 - Python 通常使用四个空格作为一级缩进
- **可迭代对象**：可以让 Python "一个接一个取出里面元素"的数据。
- **for column in X_train:** 里面的 column 默认依次是列名字符串。

2.6 函数、参数、方法与作用域

- **方法**：属于某个对象的函数。
- **参数**：交到函数括号中的数据。
- **形参**：定义函数时，先留出来的一个位置，用来表示“以后别人调用这个数据时，会把某个数据放到这里”。
- **位置参数**：根据参数出现的位置判断它应该交给谁，如Adam优化器第二个参数默认是学习率。
- **关键词参数**：如"`lr=0.001`"，直接写出参数名称。
- **def** 定义一个函数。
- 如果把函数定义到一个类里，那么这个函数就叫这个类的方法。
 - 方法是一种与类或对象关联起来的函数。
- **绑定方法**：一个类中的方法，通过某个具体对象访问后，Python 已经把这个对象和方法关联起来。
- **可调用对象**：不是普通函数，但可以像函数一样在后面写 `()` 来使用
- **作用域**：一个变量在那些哪些范围内能够被访问。
 - **for** 不会创建一个新的局部作用域，循环结束后，变量保存是最后一次循环的数据。
- **with** 用来表示在某一段代码执行期间，临时启用某种环境；这段代码结束后，再退出这个环境。

2.7 类、实例与继承

- **class (类)**：用来描述“一类对象应该有哪些数据和功能”的模板。
- **instance(实例)** 根据某个类创建出来的具体对象。
- **继承**：我们自己创建的 `TitaniModel` 在拥有自己功能的同时，也获得 `nn.Module` 已经提供的模型功能。

- `class TitanicModel(nn.Module):`
- **init()** 一个特殊函数，在创建对象时会自动执行。模型刚刚创建时，用来准备模型内部结构的地方。
- **self** 当前这个模型对象自己。
- **super().int()** 因为 `TitanicModel` 继承了 `nn.Module`，所以创建自己的模型时，需要先让 PyTorch 的 `nn.Module` 完成它自身必要的初始化。这串代码可以实现它的初始化。

2.8 解包、异常与程序入口

- **解包**：把一个含多个元素的结构位置拆给多个变量。
 - `a, b = (10, 20)` 相当于把 (10, 20) 拆开成 10、20，再分别赋值给 a、b。
- **raise** 主动告诉 Python 出现了不合法的情况，停止正常执行，并产生一个错误。
- **ValueError** Python 自带的一种异常类型。表示数据类型可能没问题，但是这个值不合法。
 - `raise ValueError("Pclass must be 1, 2 or 3")`
- **not in** 表示不在某个集合/序列中。
- **name** Python 在运行每一个 .py 文件时，会自动给它提供一个特殊变量：`name`
 - 直接运行代码时，Python 会把文件的 `__name__` 设置为 `"main"`，所以 `__name__ == "__main__"` 结果为真。
 - 如果另一个程序 `import main`，那么 `__name__` 通常是 `"main"`，此时 `__name__ == "__main__"` 结果为假。

3. Pandas

3.1 Pandas、DataFrame 与 Series

- **Pandas**：专门处理表格数据的 Python 库。
- **Pandas**：主要用来读取 CSV、清洗数据、处理缺失值、选择特征等。
- **DataFrame**：一种专门表示二维表格的数据形式。
 - `pandas.read_csv(train.csv)` 会把 `train.csv` 变成 Pandas 表格。
- **Series**：表格中的一列一维数据。
- **pandas.read_csv()**：使用 Pandas 读取 CSV。
- **data["Sex"]**：从 `data` 这张表中取出名为 `"Sex"` 的这一列。

3.2 Pandas 常用方法

- Pandas 在统计计算中通常会自动忽略 `NaN` 缺失值。
- **head()**：查看表格开头的几行数据，括号里填写行数，括号里不填写数字默认为五行。
 - `"data.head()"` 表示让 `data` 执行自己的 `head` 功能。
- **info()**：DataFrame 的一个方法，显示这张 DataFrame 的总体信息。
 - 包括索引、列名，非缺失值数量，数据类型
 - 不需要 `print()`，`info()` 本身会把信息显示到终端
- **isna()**：判断 DataFrame 中的每一个位置是不是缺失值。
- **notna()**：判断是不是非缺失值。
- **sum()**：求和。
- **unique()**：例如某一列是 `male`、`male`、`female`、`female`、`male`，执行 `unique()` 后只保留不同的值 `male`、`female`。
- **value_counts()**：统计某一列中，每一种不同的值分别出现过多少次。
 - 括号中为空，默认忽略 `NaN`。
- **sample()**：随机抽取、打乱数据。

- **mean()**: 计算均值。
- **median()**: 计算中位数。
- **mode()**: 找到一组数据的众数（一组数据中出现次数最多的值）。
- **min()**: 取最小值。
- **max()**: 取最大值。
- **fillna()**: 把缺失值NaN替换为括号内参数的值。无缺失就保持原值。
- **copy()**: 创建一份独立的数据副本。
- **astype()**: 把数据转换成指定的数据类型。
- **nunique()**: 统计有多少个不同的值。
- **describe()**: 快速统计 DataFrame 中的一些基本信息。
- **std()**: 计算标准差。
- **to_numpy**: 把DataFrame 或 Series 中的数据取出来，转换成 Numpy 数组。
 - 此时 Numpy 数组中列名会消失，只保存数字。

3.3 Pandas 顶层函数

- **pd.isna(x)** 用来判断 x 是否属于缺失值。

3.4 Pandas 属性

- **shape**: 一种属性，这张表有多少行、多少列。
- **columns**: 所有列的名称。
- **dtypes**: 查看 DataFrame 中每一列的数据类型。

3.5 Pandas 方法参数

- **dropna**: 是否把缺失值丢掉、不参与统计。
 - dropna=True 表示忽略缺失值；dropna=False 表示不忽略缺失值，算上NaN.
- **farc**: 来自 **fraction**（比例），后面 = 一个在 0 ~ 1 的数，表示抽取占比多少的数据。
- **random_state**: 固定随机操作的随机起点。
- **dtype**: 数据类型（data type）。
- **stratify**: 按照指定数据的类别比例进行分层抽样。

3.6 Pandas 索引

- - **iloc()**: integer location, 一个索引器，根据“行的位置编号”选择数据。

3.7 Pandas 之外的数据处理库和函数

- ****scikit-learn**: 可以做数据划分、预处理、传统机器学习等的库。
- **train_test_split()**: **scikit-learn** 中专门用于划分训练集和测试集地函数。
 - 基本形式: train_test_split(数据1, 数据2, test_size=..., random_state=..., stratify=...)
 - 划分出的数据集是重新构造的，与原来的数据集独立，因此不用 **.copy()**。

4. NumPy

4.1 NumPy 与 ndarray

- **NumPy**: 专门用于数值计算的 Python 包，核心数据结构是多维数组 **ndarray**（比普通 Python **list** 更适合大量数值运算的数组）。

- **NaN** Pandas/NumPy 中常见的缺失值。

4.2 Python、Numpy、PyTorch 列表//数组/张量

- **表示多个数字**

- Python list

```
[1, 2, 3]
```

- Python range

```
range(1, 4)
```

- NumPy array

```
np.array([1, 2, 3])
```

- PyTorch Tensor

```
torch.tensor([1, 2, 3])
```

5. PyTorch

5.1 PyTorch 与 Tensor

- **PyTorch**：主要用来 Tensor 数值计算、建立模型、自动求导、反向传播、参数更新、模型训练等。
- **Tensor 张量**：PyTorch 用来保存和计算数值数据的多维数组。
- **torch.tensor()**：根据已有数据创建一个 Tensor。
- **unsqueeze()** 在指定位置增加一个长度为 1 的维度。
- **clone()**: 创建当前 Tensor 数据的一份独立副本。
- **detach()**: 得到一个不参与自动求导计算图的Tensor。
- **.item()**: 从只包含一个数的 Tensor 中取出那个普通数值。
- **size ()** PyTorch 张量的第几维多长。
- **float()**: Tensor 的一个方法，可以把Tensor中的数据转换为浮点数类型。
- **torch.from_numpy(a)** 通常共享数据。
- **torch.tensor(a)** 通常复制数据。
 - Pytorch 并不专门把Pandas DataFrame 设计成**torch.tensor()**的标准输入。不同数据内容下可能出现类型、形状等问题，特别是DataFrame 里如果混有字符串、对象类型，就更不行。

5.2 随机种子

- **torch.manual_seed()**：设置 PyTorch 使用的随机种子。

5.3 Dataset、TensorDataset、DataLoader 与 Batch

- ****Dataset (数据集)** 🤖 *把“特征”和“对应标签”组装成一个个配对样本的数据结构。
- **TensorDataset**：把几个长度相同的 Tensor 按第一个维度一一配对。
- **DataLoader (数据加载器)**：从 Dataset 中按照特定方式，一批一批取出样本。
- **Batch (批次)**：一次送给模型的一小组样本。
- **shuffle**：打乱顺序。

- **shuffle=True**：打乱。
- **shuffle=False**：不打乱。

5.4 定义 PyTorch 模型

- **import torch.nn as nn**
- **层**：神经网络中的“层”可以理解成：接受一组数字，进行一组参数化计算，再输出另一组数字的计算单元。
- **nn.Linear()**：线形层，用于加权、引入偏置。

$$y = Wx + b$$

- ****nn.Linear(输入数量, 输出数量)**
- **nn.ReLU()**：ReLU 激活函数。

$$\text{ReLU}(x) = \max(0, x)$$

- **nn.Module**：PyTorch 提供的神经网络模型基础类。
- **fc** 常来自 fully connected，全连接层。
 - **self.fc1** 表示当前模型自己的 **fc1** 层
- **forward()** 表示 **前向计算**，也就是数据进入模型后，按照什么顺序进行计算，最后得到输出。

5.5 Logit、损失函数与计算图

- **logit**：二分类模型输出的一个原始分数，还没有转换成 0~1 的概率。
- **损失函数**：用一个数衡量模型当前预测和真实答案之间差得有多大。
- **nn.BCEWithLogitsLoss()** 一个类，括号左边写输出，右边写真实值。先 Sigmoid，再 BCE，然后计算损失。
- **计算图**：PyTorch 在模型做前向计算时，会记录“这个结果是经过哪些计算得到的”。这些运算关系组成一个结构，叫做计算图。
 - 作用是，后面进行反向传播时，PyTorch 能沿着这些运算关系自动计算各个参数的梯度。
- **grad_fn**：gradient function，梯度函数信息。
- **requires_grad**：PyTorch 是否需要追踪这个 Tensor 参与的运算，以便之后计算关于它的梯度。

5.6 参数、梯度与优化器

- ****parameters()**：**给出模型中所有需要训练的参数。
- **优化器**：根据参数的梯度，决定怎么样更新参数的工具。
- ****torch.optim.Adam()**Adam** 优化器。
- **zero_grad()**：优化器的方法，把模型参数上一次保存的梯度清掉。
- **.backward()**：从损失开始，沿着之前记录的计算过程反向计算梯度。
- **optimizer.step()**：让优化器根据刚刚得到的梯度，更新它负责管理的模型参数。
- **running_loss**：自己起的变量名，可以理解成当前这个 epoch 到目前为止累计的总损失。

5.7 训练、评估与关闭梯度

- **no_grad()** 表示接下来只是做预测，不需要 PyTorch 为反向传播保存梯度计算所需的信息。
- **eval.**(把模型切换到“评估状态”。

5.8 保存、加载与检查模型

- **torch.save(要保存的内容, "文件名")** 把要保存的内容保存到文件中。
 - PyTorch 项目中文件拓展名常用 .pth。
- ****torch.load()**取出文件中保存的数据, 并赋值给一个变量。
- **torch.allclose(a, b)** 检查两个 Tensor中对应位置的所有数字是否足够接近。
- **state_dict()** 返回当前模型中需要保存的参数及其数值。PyTorch 方法。
 - 一种有序的字典类型。
- **load_state_dict()** 把一个state_dict中保存的参数装入当前模型。

6. Matplotlib

- **plt.plot()** 画折线图。
 - 支持一组或者多组图像, 可以选择许多控制线条外观的参数和图例。
 - **label=** 取名。
 - **linewidth=** 粗细。
 - **linestyle=** 线型, 如实线、虚线等。
 - **maker=** 每个数据点使用什么标记。
 - **markersize=** 数据点标记大小。
 - ****"o-"** 一种简写, o表示数据点画成圆圈, -表示数据点之间使用时线连接。
- **plt.xlabel ()** 添加横坐标。
- **plt.ylabel ()** 添加纵坐标。
- **plt.title ()** 添加标题。
- **plt.legend()** 显示图例。
- **plt.savefig ()** (在某个位置)保存图片
- **plt.show()** 显示图片
- 如果在保存/显示图片之前定义了多个曲线, 保存/显示时会把两个曲线显示在一个文件中。

7. 新乘客预测

7.1 终端输入与输入检查

- **input()** 在终端显示一段提示文字, 然后等待用户输入内容。
- **raise** 主动告诉 Python出现了不合法的情况, 停止正常执行, 并产生一个错误。
- **ValueError** Python 自带的一种异常类型。表示数据类型可能没问题, 但是这个值不合法。
 - `raise ValueError("Pclass must be 1, 2 or 3")`
- **not in** 表示不在某个集合/序列中。
- `passenger[feature] is not None and passenger[feature] < 0`
 - 真的存在数值且小于零。
- **zip()** 将两个序列按位置一一对应。
 - `for feature, error_message in zip(features, error):`
 - `features, error` 是两个等长列表。
- **None** Python 常见的缺失值。
- **NaN** Pandas/NumPy 中常见的缺失值。

- **pd.isna(x)** 用来判断 x 是否属于缺失值。

7.2 预测结果的接收

- **解包**：把一个含多个元素的结构位置拆给多个变量。
 - **a, b = (10, 20)** 相当于把 (10, 20) 拆开成10、20，再分别赋值给a、b。

7.3 运行入口

- **name** Python 在运行每一个 .py 文件时，会自动给它提供一个特殊变量:**name**
 - 直接运行代码时，Python会把文件的__name__ 设置为"**main**"，所以 `__name__ == "__main__"` 结果为真。
 - 如果另一个程序 import main ,那么__name__ 通常是"main",此时 `__name__ == "__main__"` 结果为假。

8. Pandas → NumPy → PyTorch 数据流

8.1 DataFrame → NumPy → Tensor

- **DataFrame → Tensor** :
 - DataFrame：有数值、列名、列顺序。
 - → `.to_numpy()`
 - NumPy array：有数值、位置顺序，没有列名。
 - → `torch.tensor()`
 - Tensor：有数值、维度/形状、位置顺序、没有列名，不保存字符串。。

8.2 `.to_numpy()` 的作用

- **to_numpy()**：把DataFrame 或 Series 中的数据取出来，转换成 Numpy 数组。
 - 此时 Numpy 数组中列名会消失，只保存数字。
 - 常在torch.tensor后，接此方法。

8.3 `torch.tensor()` 与 `torch.from_numpy()`

- **torch.from_numpy(a)** 通常共享数据。
- **torch.tensor(a)** 通常复制数据。
 - Pytorch 并不专门把Pandas DataFrame 设计成`torch.tensor()`的标准输入。不同数据内容下可能出现类型、形状等问题，特别是DataFrame 里如果混有字符串、对象类型，就更不行。

8.4 四种“表示多个数字”的方式

- **表示多个数字**

- Python list

[1, 2, 3]

- Python range

range(1, 4)

- NumPy array

```
np.array([1, 2, 3])
```

- PyTorch Tensor

```
torch.tensor([1, 2, 3])
```

9. Git 与 Github

- **.pyc** Python 自动生成的字节码缓存文件，再次运行时可以少做一部分重复缓存工作。
 - 运行时可自动生成，不需要上传 Github.
- **pycache/** 通常储存 .pyc 文件。
- **.venv/** 忽略名字，叫 **.venv** 的文件夹。
- **venv/** 忽略名字，叫 **venv** 的文件夹
 - 这两个通常是 Python 虚拟环境文件夹。